A PROJECT REPORT

on

Ayurvedic Classification as a Rule-Based Expert System

Submitted By

PAVENDRA YADAV

in partial fulfillment for the award of

the degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

Under the guidance of

Asst. Prof. Sini.V.

Department of Computer Science & Information Technology



SMT. DEVKIBA MOHANSINHJI CHAUHAN COLLEGE OF COMMERCE AND SCIENCE

SILVASSA-UT OF DD & DNH-396230

Sem V

2026-27

Smt. Devkiba Mohansinhji Chauhan College of Commerce and Science
(Affiliated to University of Mumbai)
SILVASSA-UT OF DD & DNH-396230

Department of Computer Science & Information Technology

CERTIFICATE

This is to certify that **PAVENDRA YADAV** of T.Y.B.Sc. Computer Science **(Sem V)** class has satisfactorily completed the Project **"Ayurvedic Classification as a Rule-Based Expert System"** to be submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year 2026-2027.

Date of Submission:

Signature of Examiner

Project Guide

College Seal **Coordinator**

DECLARATION

We, **PAVENDRA YADAV** hereby declare that the project entitled **"Ayurvedic Classification as a Rule-Based Expert System "** Submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2026-27** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

We would like to convey our heartfelt gratitude to **Asst. Prof. Sini.V** for her tremendous direction and assistance in the completion of our project. We are eternally grateful to you.

We are deeply indebted to Department Coordinator **Asst. Prof. Sini.V** and our Principal **Dr. Seema Pillai** for giving us this valuable opportunity to do this project.

We are thankful to our college **Smt. Devkiba Mohansinhji Chauhan College of Commerce and Science** for considering our project and extending help at all stages needed during our work of collecting information regarding this project.

We express our hearty thanks to them for their assistance without which it would have been difficult in finishing this project synopsis and project review successfully.

We also like to thank my friends and parents for their support and encouragement as we worked on this assignment.

INDEX

Sr.	Content	Page
1	**Title**	**6**
2	**Problem Statement**	**6**
3	**Conceptual Mapping Table**	**7**
4	**Algorithm Specification**	**7**
	4.1 Formal specification	7
	4.2 Pseudo-code	8
	4.3 Sample rules	8
5	**Working Code**	**9**
	5.1 prakriti_expert_system.py	9
	5.2 Sample output	12
6	**Test Cases**	**13**
	6.1 test_prakriti.py	14
7	**Complexity and Limitations**	**17**
	7.1 Complexity	17
	7.2 Correctness argument	17
	7.3 Limitations	17
7	**Conclusion**	**18**
	References	18

1.Title

Ayurvedic Prakriti Classification as a Rule-Based Expert System

This project models the Ayurvedic assessment of an individual's constitution (Prakriti) as a rule-based expert system. Observed traits (*lakshanas*) such as body frame, skin, digestion and sleep are treated as facts, and the classical guidelines of the Charaka and Sushruta Samhitas are encoded as 30 IF-THEN production rules. A forward-chaining inference engine accumulates Vata, Pitta and Kapha scores and classifies the result as a pure, dual-dosha or balanced (Sama) constitution. An explanation module lists every rule that fired, making each decision transparent. The system is verified with 16 test cases and is intended for academic classification only, not medical diagnosis.

2.Problem Statement

Ayurveda, the classical Indian system of medicine, classifies an individual's natural constitution (*Prakriti*) into one of three *doshas*, **Vata**, **Pitta** and **Kapha**, or a combination of them. The classification is done by observing a person's traits (*lakshanas*) such as body frame, skin, digestion, sleep and temperament, and reasoning from them using guidelines recorded in texts such as the Charaka Samhita (Vimana Sthana, Ch. 8) and the Sushruta Samhita (Sharira Sthana, Ch. 4).

This project models that classical reasoning process as a **rule-based expert system**. Observed traits are represented as *facts*, the shastric guidelines are encoded as *IF-THEN production rules*, and a *forward-chaining inference engine* accumulates dosha scores and produces a Prakriti classification together with an *explanation* of which rules fired.

Scope: the system performs *classification only*. It is not a medical diagnosis or treatment tool.

Objectives

- Formalise the Prakriti-assessment process as an explicit, inspectable rule base.
- Implement a deterministic inference engine with clear conflict-resolution rules for mixed (dual-dosha) results.
- Provide an explanation module, the defining feature of an expert system.
- Verify behaviour with at least 10 test cases, including boundary and invalid inputs.

3. Conceptual Mapping Table (IKS element → CS element)

IKS element	CS element	Notes
Tridosha (Vata, Pitta, Kapha)	Output classes / score accumulators	Three counters, one per dosha
Lakshana (observed trait)	Fact (attribute-value pair)	e.g. `skin = dry`
Prakriti examination (*Pariksha*)	Working memory (fact base)	Input dictionary
Shastra's guidelines linking traits to doshas	IF-THEN production rules (knowledge base)	30 rules, R01 to R30
Relative importance of a trait (e.g. frame, *Agni*/digestion)	Rule weight	Major traits weight 2, others 1
Vaidya's reasoning	Inference engine (forward chaining)	Fires all matching rules
Mishra Prakriti (dual constitution)	Multi-label tie-breaking rule	Top two scores within a margin
Sama (balanced) Prakriti	Threshold rule on score spread	Max minus min at most 1
Vaidya explaining the assessment	Explanation module (rule trace)	Lists fired rules
Incomplete history	Input validation / insufficient-data check	At least 6 of 10 attributes needed

4. Algorithm Specification

4.1 Formal specification

- Attribute set **A** = {frame, skin, hair, appetite, digestion, sleep, temperament, climate, speech, memory} (10 attributes). Each attribute *a* has a finite domain D(a) of 3 allowed values.
- Facts **F** is a partial function F: A → values, with F(a) in D(a).
- A rule **r** = (id, attribute, value, dosha, weight). The rule base **R** has 30 rules, one per attribute-value pair.
- Rule *r* **fires** if and only if F(r.attribute) = r.value.
- Score of dosha *d*: S(d) = sum of r.weight over all fired rules with r.dosha = d.
- Constants: MARGIN = 2, TRIDOSHA_SPREAD = 1, MIN_FACTS = 6.

Decision rules (applied in this order):

1. If |F| < MIN_FACTS, the result is *Insufficient data*.
2. Else if max(S) − min(S) ≤ TRIDOSHA_SPREAD, the result is **Tridoshic (Sama)**.
3. Else if (highest − second highest) ≤ MARGIN, the result is the **dual-dosha** of the top two (named in canonical order Vata, Pitta, Kapha).
4. Else the result is the **pure** dosha with the highest score.

Score ties are broken by the canonical order Vata, Pitta, Kapha, so the output is fully deterministic.

4.2 Pseudo-code

```
ALGORITHM PrakritiClassifier(facts)
  INPUT : facts  - map attribute -> value (partial allowed)
  OUTPUT: result, scores, trace

  1.  FOR each (attr, val) in facts:        // validation
          IF attr not in KNOWLEDGE        THEN raise ERROR
          IF val  not in KNOWLEDGE[attr]  THEN raise ERROR
  2.  IF size(facts) < MIN_FACTS THEN
          RETURN "Insufficient data"
  3.  scores <- {Vata:0, Pitta:0, Kapha:0};  trace <- []
  4.  FOR each rule in RULE_BASE:           // forward chaining
          IF facts[rule.attr] == rule.value THEN
              scores[rule.dosha] <- scores[rule.dosha] + rule.weight
              append rule to trace
  5.  ranked <- doshas sorted by (score desc, canonical order)
  6.  IF scores[ranked[0]] - scores[ranked[2]] <= TRIDOSHA_SPREAD THEN
          result <- "Tridoshic (Sama)"
      ELSE IF scores[ranked[0]] - scores[ranked[1]] <= MARGIN THEN
          result <- dual(ranked[0], ranked[1])
      ELSE
          result <- pure(ranked[0])
  7.  RETURN result, scores, trace
```

4.3 Sample rules

Rule	IF	THEN
R01	frame = thin	Vata += 2
R02	frame = medium	Pitta += 2
R03	frame = broad	Kapha += 2
R04	skin = dry	Vata += 1
R05	skin = warm_sensitive	Pitta += 1
R13	digestion = gas_constipation	Vata += 2
R14	digestion = acidity_loose	Pitta += 2
R15	digestion = sluggish	Kapha += 2

The full 30-rule base is generated from the `KNOWLEDGE` table in the code.

5. Working Code

Language: Python 3 (standard library only). Files: `prakriti_expert_system.py` (engine and CLI) and `test_prakriti.py` (tests).

Run the interactive CLI: `python3 prakriti_expert_system.py`. Run a demo: `python3 prakriti_expert_system.py --demo`. Run the tests: `python3 test_prakriti.py`.

5.1 prakriti_expert_system.py

```
"""
IKS Concept as CS Concept
Ayurvedic Prakriti Classification as a Rule-Based Expert System

Components of the expert system
  1. Knowledge base   : RULES  (IF attribute = value THEN dosha += weight)
  2. Working memory   : the user's answers (facts)
  3. Inference engine : forward chaining over the rule base
  4. Explanation      : trace of every rule that fired

NOTE: This is a CLASSIFICATION tool for an academic exercise.
      It is NOT a medical diagnosis or treatment system.
"""

import sys

DOSHAS = ("Vata", "Pitta", "Kapha")      # canonical order (used for tie-breaking)
MARGIN = 2          # top-two scores within MARGIN  -> dual-dosha
TRIDOSHA_SPREAD = 1  # max - min <= this              -> Tridoshic (Sama)
MIN_FACTS = 6        # minimum answered attributes required

# Attribute -> {value: (dosha, weight)}
# Weight 2 for the two "major" attributes (body frame, digestion), 1 otherwise.
# Traits are a simplified form of the classical Prakriti lakshanas described in
# Charaka Samhita (Vimana Sthana, Ch. 8) and Sushruta Samhita (Sharira Sthana, Ch. 4).
KNOWLEDGE = {
    "frame":       {"thin": ("Vata", 2), "medium": ("Pitta", 2), "broad": ("Kapha", 2)},
    "skin":        {"dry": ("Vata", 1), "warm_sensitive": ("Pitta", 1), "thick_cool": ("Kapha", 1)},
    "hair":        {"dry_thin": ("Vata", 1), "fine_early_gray": ("Pitta", 1), "thick_oily": ("Kapha", 1)},
    "appetite":    {"irregular": ("Vata", 1), "strong_sharp": ("Pitta", 1), "slow_steady": ("Kapha", 1)},
    "digestion":   {"gas_constipation": ("Vata", 2), "acidity_loose": ("Pitta", 2), "sluggish": ("Kapha", 2)},
```

```
    "sleep":       {"light_broken": ("Vata", 1), "moderate": ("Pitta", 1),
"deep_long": ("Kapha", 1)},
    "temperament": {"anxious_quick": ("Vata", 1), "irritable_intense":
("Pitta", 1), "calm_steady": ("Kapha", 1)},
    "climate":     {"dislikes_cold": ("Vata", 1), "dislikes_heat": ("Pitta",
1), "dislikes_damp": ("Kapha", 1)},
    "speech":      {"fast_talkative": ("Vata", 1), "sharp_precise": ("Pitta",
1), "slow_measured": ("Kapha", 1)},
    "memory":      {"quick_forgets": ("Vata", 1), "sharp_moderate": ("Pitta",
1), "slow_lasting": ("Kapha", 1)},
}

# Flatten the knowledge into numbered production rules  R01 ... R30
RULES = []
for _attr, _opts in KNOWLEDGE.items():
    for _val, (_dosha, _w) in _opts.items():
        RULES.append({"id": f"R{len(RULES) + 1:02d}", "attr": _attr,
                      "value": _val, "dosha": _dosha, "weight": _w})


def validate(facts):
    """Reject unknown attributes / invalid values (input validation)."""
    if not isinstance(facts, dict):
        raise TypeError("facts must be a dict {attribute: value}")
    for attr, val in facts.items():
        if attr not in KNOWLEDGE:
            raise ValueError(f"Unknown attribute: {attr!r}")
        if val not in KNOWLEDGE[attr]:
            raise ValueError(f"Invalid value {val!r} for {attr!r}. "
                             f"Allowed: {list(KNOWLEDGE[attr])}")


def infer(facts):
    """Forward chaining: fire every rule whose condition matches the
facts."""
    scores = {d: 0 for d in DOSHAS}
    trace = []
    for rule in RULES:                                  # one pass over rule
base
        if facts.get(rule["attr"]) == rule["value"]:    # condition matches
            scores[rule["dosha"]] += rule["weight"]     # action
            trace.append(rule)
    return scores, trace


def classify(scores):
    """Conflict resolution / decision rules on the accumulated scores."""
    ranked = sorted(DOSHAS, key=lambda d: (-scores[d], DOSHAS.index(d)))
    top, second = ranked[0], ranked[1]
```

```
    hi, lo = scores[ranked[0]], scores[ranked[2]]
    if hi - lo <= TRIDOSHA_SPREAD:
        return "Tridoshic (Sama)"
    if scores[top] - scores[second] <= MARGIN:
        pair = sorted([top, second], key=DOSHAS.index)
        return f"{pair[0]}-{pair[1]} (dual)"
    return f"{top} (pure)"


def prakriti(facts):
    """Full pipeline: validate -> infer -> classify -> explain."""
    validate(facts)
    if len(facts) < MIN_FACTS:
        return {"result": "Insufficient data", "scores": None, "trace": [],
                "note": f"Need at least {MIN_FACTS} attributes, got
{len(facts)}."}
    scores, trace = infer(facts)
    return {"result": classify(scores), "scores": scores, "trace": trace,
"note": ""}


def explain(out):
    lines = [f"Result : {out['result']}"]
    if out["scores"]:
        lines.append("Scores : " + ", ".join(f"{d}={out['scores'][d]}" for d
in DOSHAS))
        lines.append("Rules fired:")
        for r in out["trace"]:
            lines.append(f"  {r['id']}: {r['attr']} = {r['value']} ->
{r['dosha']} +{r['weight']}")
    if out["note"]:
        lines.append("Note   : " + out["note"])
    return "\n".join(lines)


def interactive():
    print("=== Prakriti Expert System (classification only, not medical
advice) ===")
    facts = {}
    for attr, opts in KNOWLEDGE.items():
        names = list(opts)
        print(f"\n{attr.upper()}:")
        for i, n in enumerate(names, 1):
            print(f"  {i}. {n}")
        choice = input("Choose number (Enter to skip): ").strip()
        if choice.isdigit() and 1 <= int(choice) <= len(names):
            facts[attr] = names[int(choice) - 1]
    print("\n" + explain(prakriti(facts)))
```

```
if __name__ == "__main__":
    if "--demo" in sys.argv:
        demo = {"frame": "thin", "skin": "dry", "hair": "dry_thin",
"appetite": "irregular",
                "digestion": "gas_constipation", "sleep": "light_broken",
                "temperament": "anxious_quick", "climate": "dislikes_cold",
                "speech": "fast_talkative", "memory": "quick_forgets"}
        print(explain(prakriti(demo)))
    else:
        interactive()
```

5.2 Sample output

```
Result : Vata (pure)
Scores : Vata=6, Pitta=2, Kapha=2
Rules fired:
  R05: skin = warm_sensitive -> Pitta +1
  R09: hair = thick_oily -> Kapha +1
  R10: appetite = irregular -> Vata +1
  R13: digestion = gas_constipation -> Vata +2
  R16: sleep = light_broken -> Vata +1
  R19: temperament = anxious_quick -> Vata +1
  R22: climate = dislikes_cold -> Vata +1
  R26: speech = sharp_precise -> Pitta +1
  R30: memory = slow_lasting -> Kapha +1
```

6. Test Cases

16 test cases were written (minimum required: 10) and executed with `python3 test_prakriti.py`. In the table, scores are listed as Vata / Pitta / Kapha.

#	Scenario	Input pattern	Scores (V/P/K)	Expected result	Status
1	Pure Vata	all 10 answers point to Vata	12/0/0	Vata (pure)	PASS
2	Pure Pitta	all 10 point to Pitta	0/12/0	Pitta (pure)	PASS
3	Pure Kapha	all 10 point to Kapha	0/0/12	Kapha (pure)	PASS
4	Vata-Pitta dual	frame, digestion, sleep, appetite to V; other 6 to P	6/6/0	Vata-Pitta (dual)	PASS
5	Pitta-Kapha dual	frame, digestion, skin, hair to P; other 6 to K	0/6/6	Pitta-Kapha (dual)	PASS
6	Vata-Kapha dual (name in canonical order)	frame, digestion, skin, hair to K; other 6 to V	6/0/6	Vata-Kapha (dual)	PASS
7	Boundary: difference equals MARGIN (2)	7 vs 5	7/5/0	Vata-Pitta (dual)	PASS
8	Boundary: difference just above MARGIN (3)	7 vs 4 vs 1	7/4/1	Vata (pure)	PASS
9	Tridoshic (balanced)	all three scores equal	4/4/4	Tridoshic (Sama)	PASS
10	Conflicting / mixed traits	traits spread over all three doshas	5/4/3	Vata-Pitta (dual)	PASS
11	Partial but sufficient input	7 answers, all Vata	9/0/0	Vata (pure)	PASS
12	Insufficient input	only 4 answers	none	Insufficient data	PASS
13	Invalid value	`frame = gigantic`	none	ValueError raised	PASS
14	Unknown attribute	`height = tall`	none	ValueError raised	PASS
15	Explanation consistency	one rule fires per fact; sum of fired weights equals total score	none	invariant holds	PASS
16	Rule-base integrity	number of rules	none	exactly 30 rules	PASS

Test run result:

```
test01_pure_vata (__main__.TestPrakriti.test01_pure_vata) ... ok
test02_pure_pitta (__main__.TestPrakriti.test02_pure_pitta) ... ok
test03_pure_kapha (__main__.TestPrakriti.test03_pure_kapha) ... ok
test04_vata_pitta_dual (__main__.TestPrakriti.test04_vata_pitta_dual) ... ok
test05_pitta_kapha_dual (__main__.TestPrakriti.test05_pitta_kapha_dual) ... ok
test06_vata_kapha_dual_canonical_order (__main__.TestPrakriti.test06_vata_kapha_dual_canonical_order) ... ok
test07_boundary_margin_equal_is_dual (__main__.TestPrakriti.test07_boundary_margin_equal_is_dual) ... ok
test08_just_beyond_margin_is_pure (__main__.TestPrakriti.test08_just_beyond_margin_is_pure) ... ok
test09_tridoshic (__main__.TestPrakriti.test09_tridoshic) ... ok
test10_conflicting_traits (__main__.TestPrakriti.test10_conflicting_traits) ... ok
test11_partial_but_sufficient_input (__main__.TestPrakriti.test11_partial_but_sufficient_input) ... ok
test12_insufficient_input (__main__.TestPrakriti.test12_insufficient_input) ... ok
test13_invalid_value_rejected (__main__.TestPrakriti.test13_invalid_value_rejected) ... ok
test14_unknown_attribute_rejected (__main__.TestPrakriti.test14_unknown_attribute_rejected) ... ok
test15_explanation_trace_matches_facts (__main__.TestPrakriti.test15_explanation_trace_matches_facts) ... ok
test16_rule_base_size (__main__.TestPrakriti.test16_rule_base_size) ... ok

----------------------------------------------------------------------
Ran 16 tests in 0.470s

OK
```

6.1 test_prakriti.py

```
import unittest
from prakriti_expert_system import prakriti, KNOWLEDGE, RULES

def build(**dosha_by_attr):
    """Helper: build facts from {attribute: 'V'|'P'|'K'}."""
    idx = {"V": 0, "P": 1, "K": 2}
    facts = {}
    for attr, d in dosha_by_attr.items():
        facts[attr] = list(KNOWLEDGE[attr])[idx[d]]
    return facts

ALL = ["frame", "skin", "hair", "appetite", "digestion",
       "sleep", "temperament", "climate", "speech", "memory"]

def mix(v=(), p=(), k=()):
    m = {}
    m.update({a: "V" for a in v}); m.update({a: "P" for a in p});
m.update({a: "K" for a in k})
    assert sorted(m) == sorted(ALL)
    return build(**m)

class TestPrakriti(unittest.TestCase):
    def check(self, facts, result, scores=None):
        out = prakriti(facts)
        self.assertEqual(out["result"], result)
        if scores:
            self.assertEqual(out["scores"], scores)
        return out

    def test01_pure_vata(self):
        self.check(mix(v=ALL), "Vata (pure)", {"Vata": 12, "Pitta": 0,
"Kapha": 0})
    def test02_pure_pitta(self):
        self.check(mix(p=ALL), "Pitta (pure)", {"Vata": 0, "Pitta": 12,
"Kapha": 0})
    def test03_pure_kapha(self):
        self.check(mix(k=ALL), "Kapha (pure)", {"Vata": 0, "Pitta": 0,
"Kapha": 12})
    def test04_vata_pitta_dual(self):
        f = mix(v=["frame", "digestion", "sleep", "appetite"],
                p=["skin", "hair", "temperament", "climate", "speech",
"memory"])
        self.check(f, "Vata-Pitta (dual)", {"Vata": 6, "Pitta": 6, "Kapha":
0})
    def test05_pitta_kapha_dual(self):
        f = mix(p=["frame", "digestion", "skin", "hair"],
                k=["appetite", "sleep", "temperament", "climate", "speech",
"memory"])
```

```
        self.check(f, "Pitta-Kapha (dual)", {"Vata": 0, "Pitta": 6, "Kapha":
6})
    def test06_vata_kapha_dual_canonical_order(self):
        f = mix(k=["frame", "digestion", "skin", "hair"],
                v=["appetite", "sleep", "temperament", "climate", "speech",
"memory"])
        self.check(f, "Vata-Kapha (dual)", {"Vata": 6, "Pitta": 0, "Kapha":
6})
    def test07_boundary_margin_equal_is_dual(self):          # 7 vs 5, diff
== MARGIN
        f = mix(v=["frame", "digestion", "skin", "hair", "sleep"],
                p=["appetite", "temperament", "climate", "speech", "memory"])
        self.check(f, "Vata-Pitta (dual)", {"Vata": 7, "Pitta": 5, "Kapha":
0})
    def test08_just_beyond_margin_is_pure(self):             # 7 vs 4, diff 3
> MARGIN
        f = mix(v=["frame", "digestion", "skin", "hair", "sleep"],
                p=["appetite", "temperament", "climate", "speech"],
k=["memory"])
        self.check(f, "Vata (pure)", {"Vata": 7, "Pitta": 4, "Kapha": 1})
    def test09_tridoshic(self):                              # 4 / 4 / 4
        f = mix(v=["frame", "skin", "hair"],
                p=["digestion", "appetite", "sleep"],
                k=["temperament", "climate", "speech", "memory"])
        self.check(f, "Tridoshic (Sama)", {"Vata": 4, "Pitta": 4, "Kapha":
4})
    def test10_conflicting_traits(self):                     # 5 / 4 / 3
        f = mix(v=["frame", "skin", "hair", "sleep"],
                p=["digestion", "appetite", "temperament"],
                k=["climate", "speech", "memory"])
        self.check(f, "Vata-Pitta (dual)", {"Vata": 5, "Pitta": 4, "Kapha":
3})
    def test11_partial_but_sufficient_input(self):           # 7 answered
        f = build(frame="V", digestion="V", skin="V", hair="V", sleep="V",
                  appetite="V", speech="V")
        self.check(f, "Vata (pure)", {"Vata": 9, "Pitta": 0, "Kapha": 0})
    def test12_insufficient_input(self):                     # only 4
answered
        out = self.check(build(frame="V", skin="P", hair="K", sleep="V"),
"Insufficient data")
        self.assertIsNone(out["scores"])
    def test13_invalid_value_rejected(self):
        with self.assertRaises(ValueError):
            prakriti({"frame": "gigantic"})
    def test14_unknown_attribute_rejected(self):
        with self.assertRaises(ValueError):
            prakriti({"height": "tall"})
    def test15_explanation_trace_matches_facts(self):        # one rule fires
per fact
        f = mix(v=ALL)
```

```
        out = prakriti(f)
        self.assertEqual(len(out["trace"]), len(f))
        self.assertEqual(sum(r["weight"] for r in out["trace"]),
sum(out["scores"].values()))
    def test16_rule_base_size(self):
        self.assertEqual(len(RULES), 30)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

7. Complexity and Limitations

7.1 Complexity

Let n = number of answered attributes, R = number of rules (30), D = number of doshas (3).

Step	Time	Space
Validation	O(n)	O(1)
Inference (scan of the rule base)	O(R)	O(R) for the trace
Classification (sort of 3 scores)	O(D log D) = O(1)	O(D) = O(1)
Total	**O(R + n)**	**O(R)**

Because R and D are constants, the practical cost per classification is constant and finishes in well under a millisecond. If the rule base grew, indexing rules by (attribute, value) in a hash map would reduce inference to O(n).

7.2 Correctness argument

- **Determinism:** rules are evaluated in a fixed order and score ties are broken by canonical order, so the same input always gives the same output.
- **Termination:** forward chaining makes a single pass over a finite rule base and fires no new rules, so the algorithm always terminates.
- **Exhaustive decisions:** the four decision branches (insufficient data, tridoshic, dual, pure) are checked in a fixed order, so every valid input receives exactly one label.
- **Consistency check:** test 15 verifies that the sum of weights of fired rules equals the total score, and that exactly one rule fires per answered attribute.

7.3 Limitations

1. **Not medical advice.** The system classifies; it does not diagnose or treat. Real Prakriti assessment is done by a trained Vaidya using observation, pulse examination and history.
2. **Simplified knowledge base.** The 10 attributes and 30 rules are a simplified, representative subset of the classical lakshanas, not the full textual corpus. Different texts and schools list different traits.
3. **Weights and thresholds are design choices.** The weights (1 or 2), MARGIN = 2 and TRIDOSHA_SPREAD = 1 are the author's choices and are not specified by the classical texts. They would need validation by domain experts.
4. **Subjective inputs.** Answers such as "irritable" or "light sleep" depend on self-assessment, so output quality depends on honest, accurate input.
5. **Hard categories.** Each trait is forced into one of three values, whereas real traits are often continuous or mixed.
6. **No Vikriti.** Ayurveda separates the inborn constitution (Prakriti) from the current imbalance (Vikriti). This system does not model Vikriti, age, season or region.
7. **Flat rule structure.** Rules are independent additive scores. There are no rule chains, no uncertainty handling (such as certainty factors) and no learning from data.

8. Conclusion

This project shows that the reasoning pattern of Ayurvedic Prakriti assessment maps cleanly onto the classical architecture of a rule-based expert system: lakshanas become facts, shastric guidelines become production rules, the Vaidya's reasoning becomes a forward-chaining inference engine, and the Vaidya's justification becomes an explanation trace. The resulting system is deterministic, explainable, and verified by 16 passing test cases covering pure, dual and balanced constitutions, boundary conditions, partial input and invalid input.

The work also highlights the limits of formalising traditional knowledge: weights, thresholds and discrete categories are modelling choices, and the output is a classification aid for study, not a clinical tool. Even so, the exercise demonstrates that Indian Knowledge Systems contain structured, rule-like reasoning that can be expressed rigorously in computational terms.

References

1. Charaka Samhita, Vimana Sthana, Chapter 8 (includes Prakriti).
2. Sushruta Samhita, Sharira Sthana, Chapter 4.
3. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach* (rule-based systems and forward chaining).